



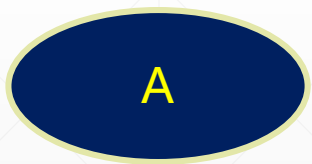
对象复制

北京理工大学计算机学院
金旭亮

概述

- 所谓“对象复制”，是指这样的一种情景：

我已经有一个对象A了，我希望把它克隆N份，得到N个与A内容一模一样的对象。



一个A对象



N个“长得一模一样”的A对象

编程实现对象的复制

编写一个简单的类A

```
class A    {  
    public int i = 100;  
}
```

ShallowCopy.java

克隆简单对象(ShallowCopy.java)

```
public static A cloneObject(A obj) {  
    A newObj = new A();  
    newObj.i = obj.i;  
    return newObj;  
}
```

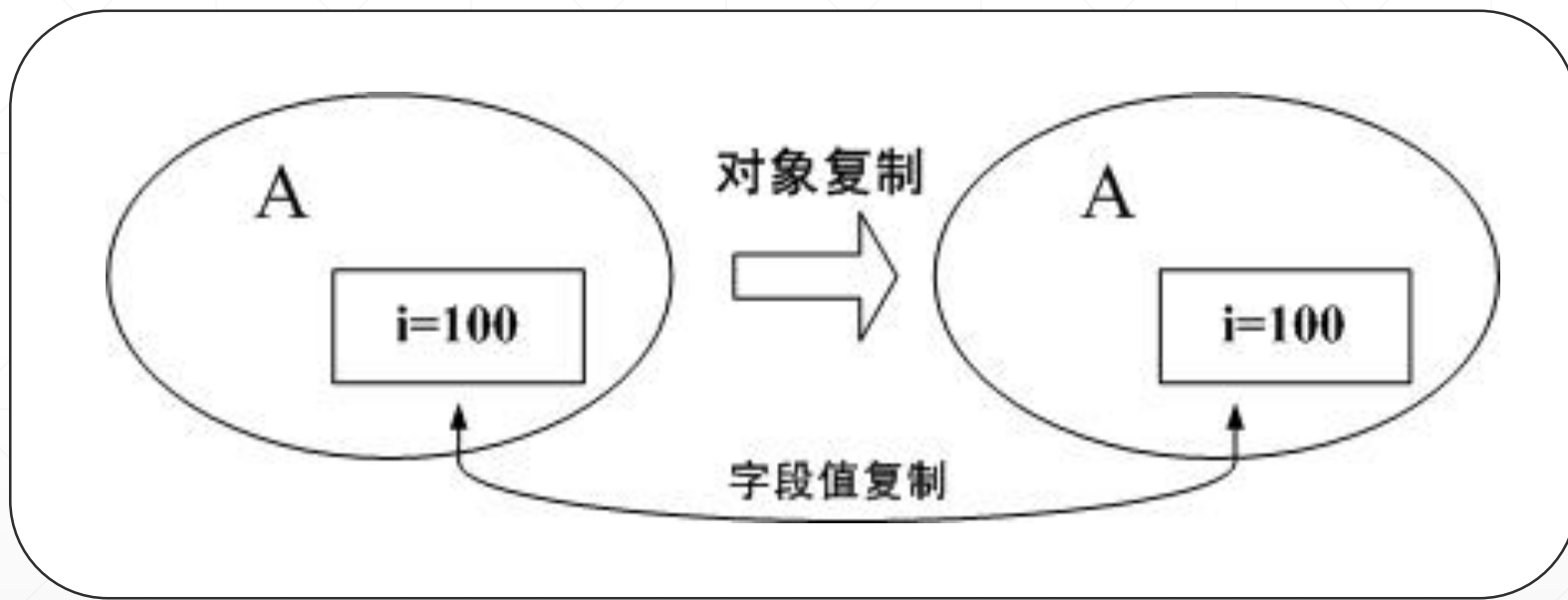
完成克隆对象
任务的方法

```
A a = new A();  
// 开始克隆  
A other = cloneObject(a);
```

克隆对象

对象的浅复制方式

前面的示例代码中，对象的复制其实如下图所示：



像这样，把一个对象的所有字段值，逐个地复制到另一个对象的对应字段，这种对象复制方式称为“**浅复制 (shallow copy)**”。

组合对象的复制问题

对于组合对象，浅复制方式会带来一些问题。请看以下示例代码，注意到A对象包容一个B对象：

```
class A {  
    public int i=100;  
    public B b;           //A包容一个B的对象  
    public A() {  
        b = new B();      //创建被包容对象  
    }  
}  
  
class B {  
    public int j = 200;  
}
```

ShallowCopy2.java

克隆组合对象(ShallowCopy2.java)

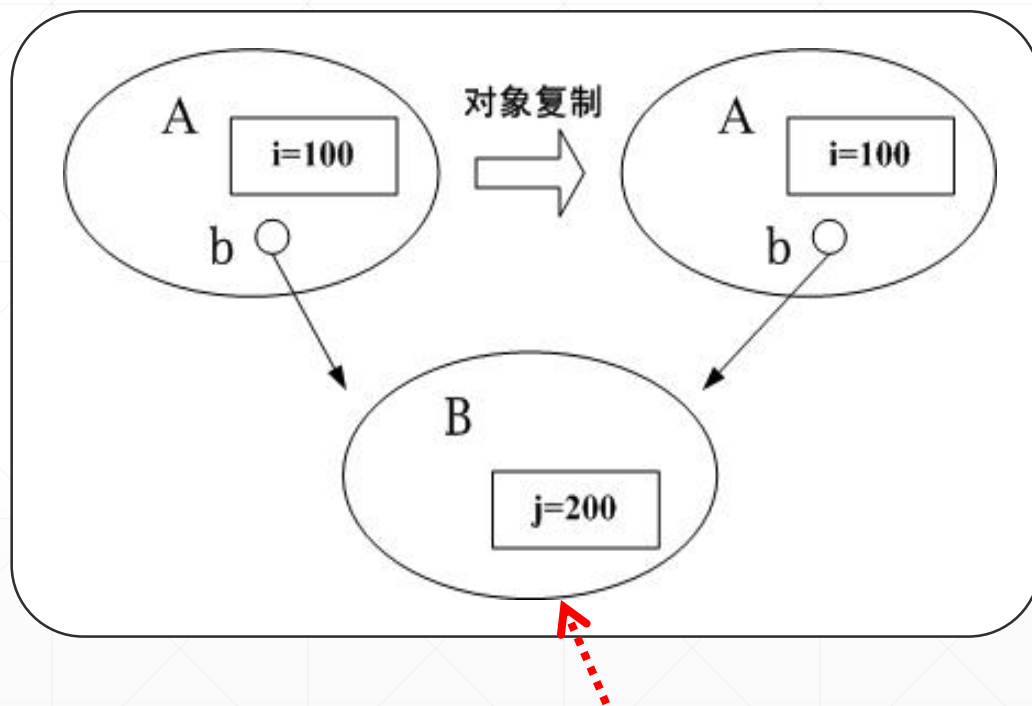
```
static A cloneObject(A obj)    {  
    A newObj = new A();  
    newObj.i = obj.i;  
    newObj.b = obj.b;  
    return newObj;  
}
```

用于完成克隆对象工作的方法

```
A a = new A();  
A other = cloneObject(a);
```

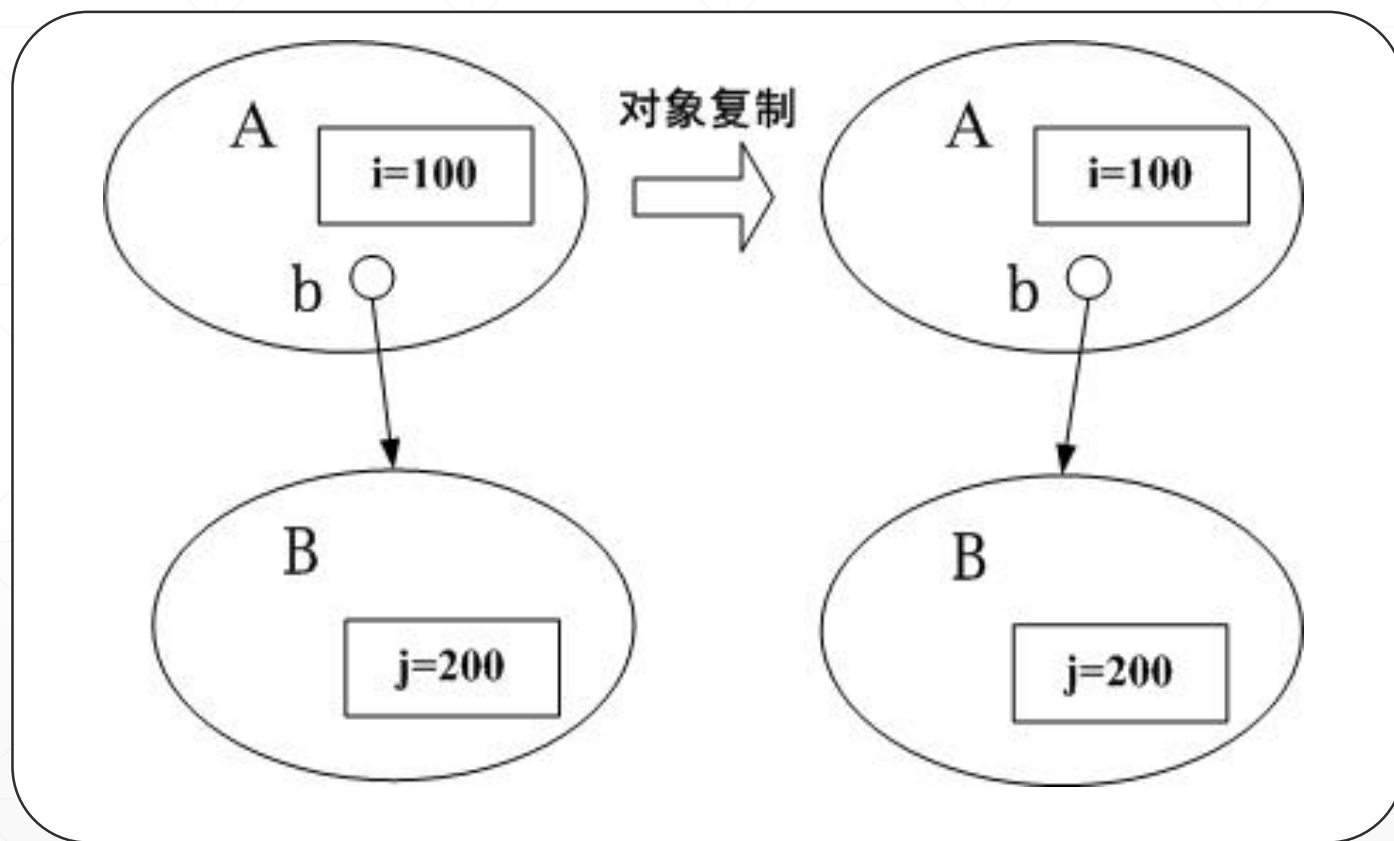
克隆对象

浅复制带来的问题



对象B没有被复制，而是被克隆前后的两个对象所“共享”，这并不符合“对象复制”的本意。我们希望完成的是“对象克隆”，即得到两个“一模一样的”，并且是“完全独立”的对象。

“深复制”——真正克隆对象



```
static A cloneObject(A obj) {  
    A newObj = new A();  
    newObj.i = obj.i;  
    //创建一个被包容的内部对象  
    newObj.b = new B();  
    newObj.b.j = obj.b.j;  
    return newObj;  
}
```

DeepCopy.java 示例实现了对象“深复制”。

将克隆功能嵌入到对象内部

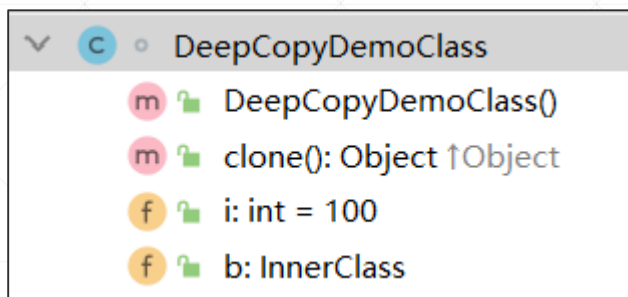
- Object类提供了一个protected clone()方法，子类可将其定义为public的，从而向外界提供克隆自己的功能：

```
protected Object clone()  
    throws CloneNotSupportedException
```



示例DeepCopy2.java，正是采用的这种方法。

手工重写clone方法实现“深复制”



DeepCopy2.java

```
class DeepCopyDemoClass implements Cloneable {  
  
    public int i = 100;  
    public InnerClass b;           //A包容一个B的对象  
  
    public DeepCopyDemoClass() {  
        b = new InnerClass();    //创建被包容对象  
    }  
    //重写基类的clone方法  
    public Object clone(){  
        var newObj = new DeepCopyDemoClass();  
        newObj.i = this.i;  
        newObj.b = new InnerClass();  
        newObj.b.j=this.b.j;  
        return newObj;  
    }  
}
```

Cloneable接口



JDK中提供了一个Cloneable接口，需要实现深复制的对象应该实现这一接口。

```
public interface Cloneable {  
}
```



神奇的是，上面这个接口根本就没有定义任何一个成员。



像这种根本就没有定义任何一个成员的接口，称为“**标记接口**”。

Cloneable接口的使用规范



依据JDK文档，一个对象选择实现Cloneable接口，必须重写Object类的clone方法，并把它改写为public的（正如我们在DeepCopy2.java中所看到的那样）。

动手动脑



- 请阅读CloneTest.java示例，它展示了Cloneable接口的具体使用方法
- 阅读提示：
 1. Employee类内部包容一个Date对象，而Date对象也实现了Cloneable接口。
 2. 注意查看一下Date对象的equals方法。
- 自我练习：

给Employee类添加equals方法，实现按值比较两个Employee对象。

小结



- 到本讲为止，我们已经介绍了三种常见面向对象编程场景：对象比较、对象组合和对象复制。
- 这三种场景在实际开发中经常出现，大家在学习时一定要注意把握其要点。
- 掌握这三种面向对象编程技巧，是进一步学习更复杂的（比如“设计模式”）编程技巧的基础。